

OpenClaw Garbage Sorting

OpenClaw Garbage Sorting

1. Course Content
 - Learning Objectives
2. Preparation
3. Robotic Arm Garbage Sorting Case
4. yolov26-OBB Garbage Recognition Node
5. Source Code Analysis
 - 5.1 GetWasteRecognitionResults — Get Waste Recognition Results
 - 5.2 GraspWaste — Grasp Target Waste
 - 5.3 PlaceWaste — Place Waste
 - 5.6 waste_classify Detection Node — YOLO Inference Thread
6. Parameter Tuning
 - 6.1 Garbage Sorting Grasp Parameters
 - 6.2 YOLO Model Detection Parameters

1. Course Content

Learning Objectives

- Understand the application and configuration of the yolov26-OBB model in garbage detection
- Learn to use GetWasteRecognitionResults, GraspWaste, PlaceWaste tools to complete garbage sorting tasks
- Master the adjustment of key parameters such as waste_grasp_offset and waste_grasp_distance to optimize grasping accuracy

2. Preparation

- Start the MicroROS chassis agent (no need to restart if already started)

```
sh start_agent.sh
```

```
mircoROS_Agent
[1777521253.413774] info | ProxyClient.cpp | create_datareader | d
subscriber created | client_key: 0x5D94732B, subscriber_id: 0x002(4), partici
pant_id: 0x000(1)
[1777521253.41761] info | ProxyClient.cpp | create_topic | t
opac created | client_key: 0x5D94732B, topic_id: 0x008(2), participant_
id: 0x000(1)
[1777521253.421603] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x5D94732B, subscriber_id: 0x003(4), partici
pant_id: 0x000(1)
[1777521253.426232] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x5D94732B, datareader_id: 0x003(6), subscri
ber_id: 0x003(4)
[1777521253.429805] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x5D94732B, topic_id: 0x009(2), participant_
id: 0x000(1)
[1777521253.433063] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x5D94732B, subscriber_id: 0x004(4), partici
pant_id: 0x000(1)
[1777521253.437859] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x5D94732B, datareader_id: 0x004(6), subscri
ber_id: 0x004(4)
```

- Start odometry, tf, robotic arm assistance, camera nodes, etc.

```
ros2 launch m3pro_bringup car_base.launch.py
```

- Start the MCP service

```
ros2 launch m3pro_bringup mcp_service.launch.py
```

- If you need voice reply, you need to start openclaw_bridge additionally, otherwise it can be omitted

```
ros2 run multi_brains_pre openclaw_bridge
```

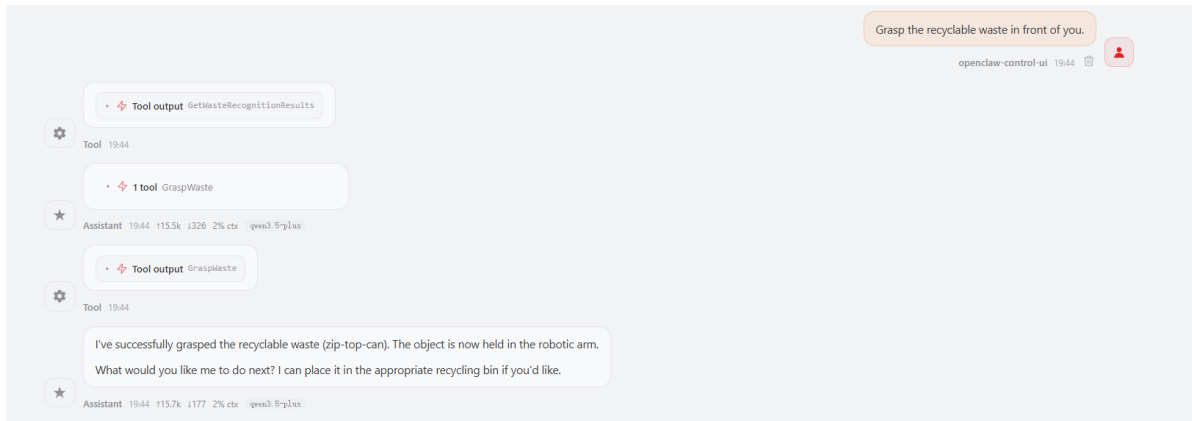
3. Robotic Arm Garbage Sorting Case

[!TIP]

- You can write commands according to your own needs. The following is a case demonstration. Choose any interaction method. Here we take the web-based WebChat as an example.

Tool	Parameters	Function
GetWasteRecognitionResults	None	Get YOLO model garbage recognition results
GraspWaste	--waste-name	Grasp waste object
PlaceWaste	None	Place waste object

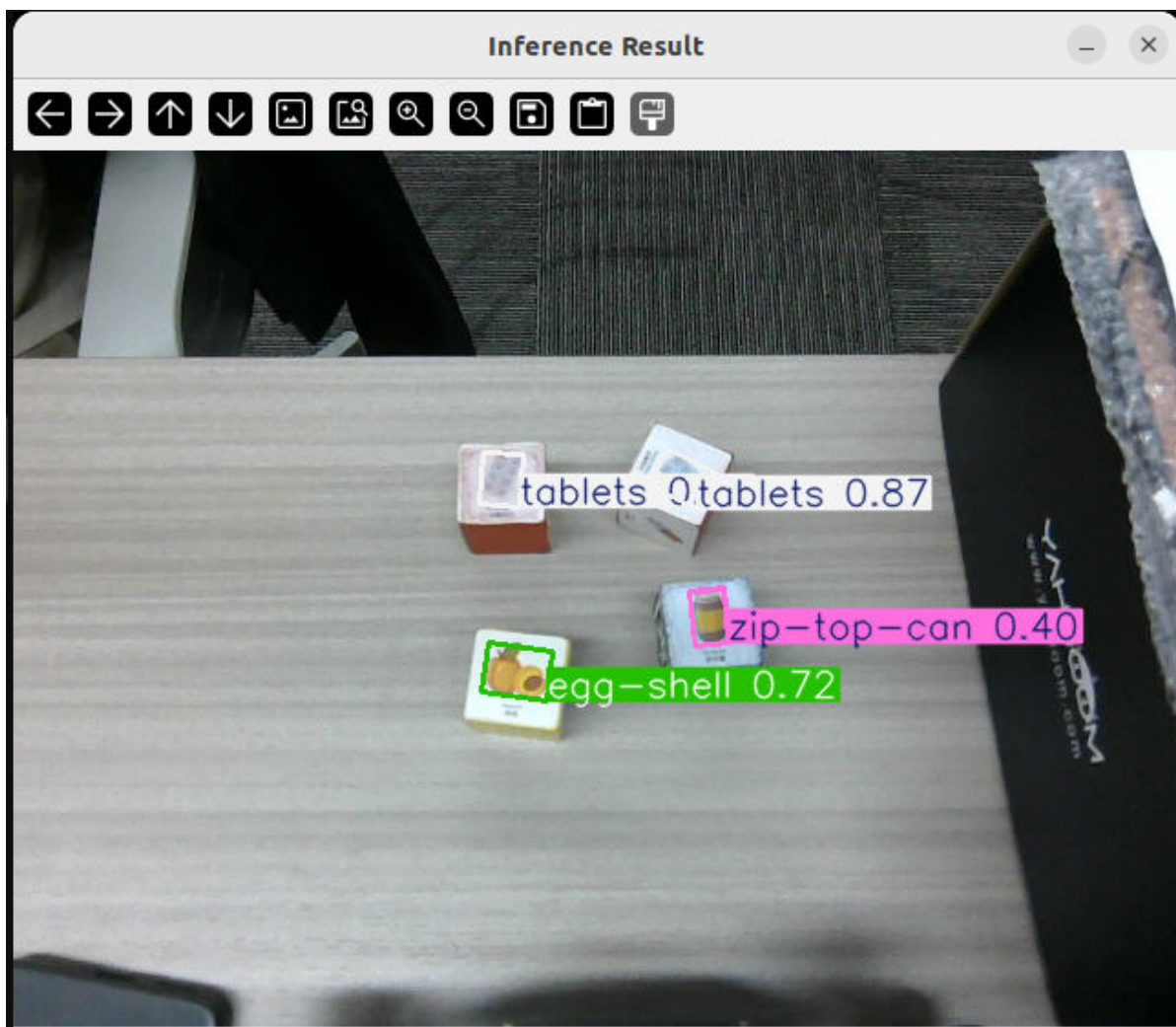
- Grasp the recyclable waste in front of you (you can also directly specify a specific waste)



- Garbage recognition here is performed using the yolov26-OBb model. The first startup has a cold start delay, requiring the yolo model to be loaded first.
 - After loading, the model automatically unloads after 240 seconds to save memory and CPU resources (configurable duration)

```
[waste_detect-6] [INFO] [1778040576.537718883] [waste_detect]: loading yolo model...
```

- After the model is loaded, the garbage detection window automatically starts, displaying the recognition results



- OpenClaw first obtains the yolo garbage recognition results through GetWasteRecognitionResults. When obtaining the results, the recognition frame image is automatically saved to the path:

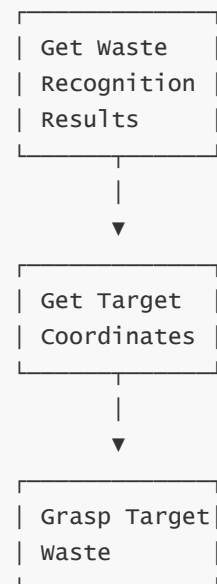
```
$HOME/M3Pro_ws/multi_brains_file/yolo_detect.png
```

- We can also directly view the garbage detection results through the robot-control CLI (suitable for debugging)

```
robot_control call-tool GetWasteRecognitionResults
```

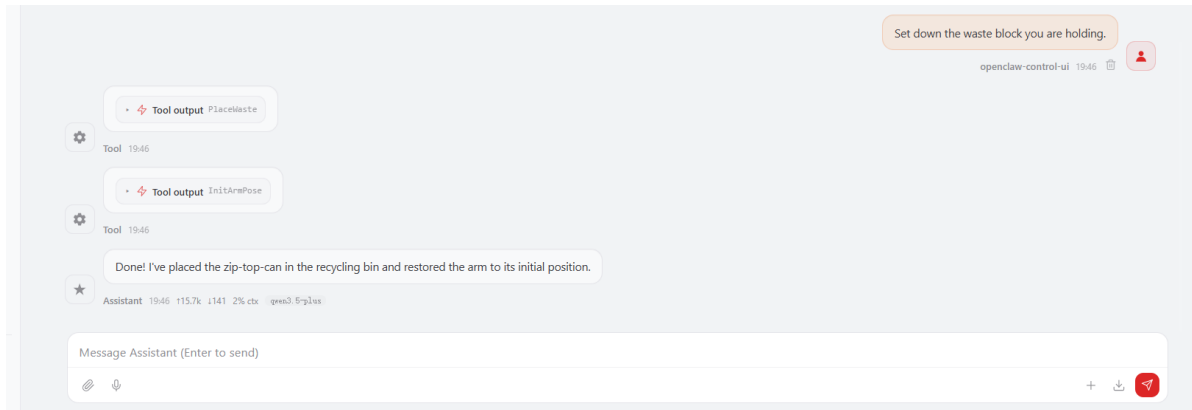
```
jetson@yahboom:~$ robot_control call-tool GetWasteRecognitionResults
{
  "result": {
    "detections": [
      "1. egg-shell (FoodWaste), confidence: 0.92",
      "2. tablets (HazardousWaste), confidence: 0.82",
      "3. tablets (HazardousWaste), confidence: 0.61",
      "4. zip-top-can (RecyclableWaste), confidence: 0.51",
      "5. disposable-chopsticks (OtherWaste), confidence: 0.23"
    ],
    "total_count": 5,
    "yolo_detect_image": "/home/jetson/M3Pro_ws/multi_brains_file/yolo_detect.png"
  }
}
```

- Then OpenClaw calls the GraspWaste tool to grasp the target waste block



- Target grasping skills
 - OpenClaw controls the robot to grasp the target object by first reading the robotic arm grasping and garbage sorting skills, following the SOP workflow
 - Skill reference: [08-openclaw skills development]

- Waste placement: to adapt to different trash bins, the placement tool uses action groups to complete the placement action (can be customized through configuration parameters)



4. yolov26-OBB Garbage Recognition Node

- Garbage detection is an independent program node. When starting the MCP service, the garbage detection node starts automatically. To reduce CPU and memory usage, detection is disabled by default at startup

```
waste_classify=Node(
    package="waste_classify",
    executable="waste_detect",
    name="waste_detect",
    parameters=[common_param_file,
                {'image_save_path':
os.path.join(os.path.expanduser('~'), 'M3Pro_ws', 'multi_brains_file', 'yolo_detect
.png')}],
                {'model_path': yolov26_model_path},
                {'show_image': True},
                {'detect_enable': False},
                {'device': device}
    ],
    emulate_tty=True,
    additional_env={'PYTHONPATH': venv_pythonpath + os.pathsep +
original_pythonpath
    },
    output='screen')
```

- After the MCP service node starts, we can manually control the start/stop of the detection function through ROS services (suitable for debugging). The waste_classify node provides the following three services
- Query current status (operation=0)

```
ros2 service call /waste_enable_srv waste_classify_msg/srv/WasteEnable "
{operation: 0}"
```

```
jetson@yahboom:~$ ros2 service call /waste_enable_srv waste_classify_msg/srv/WasteEnable "{operation: 0}"
requester: making request: waste_classify_msg.srv.WasteEnable_Request(operation=0)

response:
waste_classify_msg.srv.WasteEnable_Response(success=True, detect_enable=False, message='Current detection status: disabled')
```

- Enable garbage detection (operation=1)

```
ros2 service call /waste_enable_srv waste_classify_msg/srv/WasteEnable "{operation: 1}"
```

```
jetson@yahboom:~$ ros2 service call /waste_enable_srv waste_classify_msg/srv/WasteEnable "{operation: 1}"
requester: making request: waste_classify_msg.srv.WasteEnable_Request(operation=1)

response:
waste_classify_msg.srv.WasteEnable_Response(success=True, detect_enable=True, message='Detection enabled')
```

- The waste_classify node publishes detection results to the /waste_detections topic during detection. You can view topic data with the following command

```
ros2 topic echo /waste_detections --once
```

- Reference topic data

```
header:
  stamp:
    sec: 1778052123
    nanosec: 473421724
  frame_id: ''
waste_detections:
- class_name: peach-pit
  classification: Otherwaste
  confidence: 0.8850700259208679
  center_x: 390.0876770019531
  center_y: 314.08135986328125
  width: 32.05992126464844
  height: 25.9763240814209
  angle: -0.02274312824010849
  points:
  - x: 406.40887451171875
    y: 326.7016296386719
  - x: 405.818115234375
    y: 300.7320251464844
  - x: 373.7664794921875
    y: 301.4610900878906
  - x: 374.35723876953125
    y: 327.4306945800781
```

Topic Data Field Descriptions:

Field	Type	Description
<code>header.stamp</code>	time	Timestamp of detection result (sec: seconds, nanosec: nanoseconds)
<code>waste_detections</code>	WasteItem[]	List of detected waste objects (may contain multiple targets)
<code>class_name</code>	string	Specific category name of the waste (e.g., peach-pit, banana-peel, etc.)
<code>classification</code>	string	Waste classification type (Recyclable, KitchenWaste, OtherWaste, HazardousWaste)
<code>confidence</code>	float	Detection confidence (0.0~1.0), higher values indicate the model is more certain
<code>center_x</code>	float	X coordinate of detection box center (pixel units, relative to image top-left)
<code>center_y</code>	float	Y coordinate of detection box center (pixel units, relative to image top-left)
<code>width</code>	float	Width of detection box (pixel units)
<code>height</code>	float	Height of detection box (pixel units)
<code>angle</code>	float	Rotation angle of detection box (radians), OBB model specific parameter indicating object orientation
<code>points</code>	Point2D[]	4 vertex coordinates of the rotated box (clockwise or counter-clockwise), used for precise object boundary description

Key Concept Descriptions:

1. **OBB (Oriented Bounding Box):** Unlike traditional rectangular boxes, OBB can rotate to better fit the actual shape of objects, especially suitable for irregular objects
2. **Angle:** Indicates the rotation angle of the object relative to the horizontal direction. **The robotic arm adjusts the gripper direction based on this angle during grasping** to ensure optimal grasping
3. **Points (Vertex Coordinates):** The 4 vertices precisely describe the boundary of the rotated box, which can be used to calculate the actual size and position of the object

- Disable detection (operation=2)

```
ros2 service call /waste_enable_srv waste_classify_msg/srv/WasteEnable "{operation: 2}"
```

```
jetson@yahboom:~$ ros2 service call /waste_enable_srv waste_classify_msg/srv/WasteEnable "{operation: 2}"
requester: making request: waste_classify_msg.srv.WasteEnable_Request(operation=2)

response:
waste_classify_msg.srv.WasteEnable_Response(success=True, detect_enable=False, message='Detection disabled')
```

5. Source Code Analysis

- Source path

```
$HOME/M3Pro_ws/src/multi_brains_pre/multi_brains_pre/action_controller.py
```

5.1 GetWasteRecognitionResults — Get Waste Recognition Results

Function Description: Calls the YOLO model for garbage detection, obtaining recognition results for all waste in the current view, including category, confidence, position, and other information.

```
def GetWasteRecognitionResults(self) -> list:
    '''Get YOLO waste recognition results'''
    from multi_brains_pre.message.mcp_message import waste_recognition_result
    if not self.waste_enable_client.wait_for_service(timeout_sec=8.0) or not
self.get_waste_client.wait_for_service(timeout_sec=8.0):
        msg = "GetWaste service not available"
        self.get_logger().error(Fore.YELLOW + msg + Fore.RESET)
        return [False, msg]

    self._pubSix_Arm([90,90,50,-30,90,0])
    time.sleep(3.0)

    #TODO: waste detection causes high CPU usage; enable detection only when
    needed, and disable after getting results
    response:WasteEnable.Response =
self.waste_enable_client.call(WasteEnable.Request(operation=0))
    if not response.detect_enable:
        response:WasteEnable.Response =
self.waste_enable_client.call(WasteEnable.Request(operation=1)) #enable waste
    detection
        if not response.success:
            msg = "Failed to enable waste detection"
            self.get_logger().error(Fore.RED + msg + Fore.RESET)
            return [False, msg]
        time.sleep(10.0)#wait for first waste detect

    response:GetWaste.Response=self.get_waste_client.call(GetWaste.Request())
    if not response.success:
        msg = response.message
        self.get_logger().error(Fore.RED + msg + Fore.RESET)
        return [False, msg]

    if len(response.waste_detections) == 0:
        msg = "No waste detected"
        self.get_logger().warn(Fore.YELLOW + msg + Fore.RESET)
        return [False, msg]

    result_lines = []
    for i, item in enumerate(response.waste_detections, 1):
        result_lines.append(
            f"{i}. {item.class_name} ({item.classification}), "
            f"confidence: {item.confidence:.2f}")
```

```

result = waste_recognition_result(
    detections=result_lines,
    total_count=len(response.waste_detections),
    yolo_detect_image=response.image_path)
return [True, result]

```

Process Description:

1. **Service Check:** Wait for `waste_enable_srv` and `get_waste_srv` services to be available (8 second timeout)
2. **Adjust Detection Pose:** Call `_pubSrv_Arm([90,90,50,-30,90,0])` to move the robotic arm to the predefined detection pose, wait 3 seconds for stabilization
3. **Start/Stop Detection:**
 - Query current detection status (operation=0)
 - If not started, enable detection (operation=1), wait 10 seconds for first detection to complete
4. **Get Results:** Call `get_waste_srv` service to obtain detection results
5. **Result Processing:** Iterate through all detected waste, format output (category, classification, confidence)
6. **Return Results:** Return structured results containing detection list, total count, and recognition image path

Notes:

- ⚠ Garbage detection causes high CPU usage, so an "on-demand start/stop" strategy is adopted. YOLO detection automatically shuts down after a period of time after being enabled
- ⚠ After enabling detection, wait sufficient time (10 seconds) to ensure the model completes its first inference
- ⚠ The robotic arm must move to the detection pose first to obtain accurate recognition results

5.2 GraspWaste — Grasp Target Waste

```

def GraspWaste(self, waste_name: str) -> list[bool, str]:
    '''Grasp waste object by name'''
    self._pubSrv_Arm(self.waste_detect_pose)
    time.sleep(3.0)
    result = self._get_target_pose(waste_name, max_attempts=5)
    if not result[0]:
        return [False, result[1]]

    target_pose: Pose = result[1]
    target_item: WasteItem = result[2]

    # Adjust robot position if needed
    adjusted_flag = False
    if target_pose.position.x > self.waste_grasp_distance[0]:
        adjusted_flag = True
        delta_x = target_pose.position.x - self.waste_grasp_distance[0]
        self.move_by_odom(distance=abs(delta_x), direction='forward')
        self.get_logger().info("adjust x position finish")
        time.sleep(4.0) #wait for camera to stabilize after movement

```

```

if abs(target_pose.position.y) > self.waste_grasp_distance[1]:
    adjusted_flag = True
    delta_y = target_pose.position.y
    direction = 'left' if delta_y > 0 else 'right'
    self.move_by_odom(distance=abs(delta_y), direction=direction)
    self.get_logger().info("adjust y position finish")
    time.sleep(4.0) #wait for camera to stabilize after movement

# Re-acquire target pose after adjustment
if adjusted_flag:
    result = self._get_target_pose(waste_name, max_attempts=5)
    if not result[0]:
        return [False, result[1]]

    target_pose = result[1]
    target_item: WasteItem = result[2]

    item_rotation_angle=math.degrees(target_item.angle)
    self.get_logger().debug(f"target_item_angle: {item_rotation_angle}")
    if abs(item_rotation_angle) > 5:
        #TODO: 90 degrees is the neutral position of the end effector.
        arm_joint5_angle = 90+math.degrees(target_item.angle)
    else:
        arm_joint5_angle=90

    self._waste_grasp(grasp_x=target_pose.position.x,
                      grasp_y=target_pose.position.y,
                      grasp_z=target_pose.position.z,
                      joint_5_angle=arm_joint5_angle,
                      grasp_pitch=self.waste_grasp_offset[3])

    self.move_by_odom(direction='backward', distance=self.auto_back_dist)
    return [True, "Grasp waste successfully"]

```

Process Description:

1. **Move to Detection Pose:** Call `_pubSix_Arm(self.waste_detect_pose)` to move the robotic arm to the predefined detection pose, wait 3 seconds
2. **Get Target 3D Pose:** Call `_get_target_pose()` to get the target waste's 3D coordinates, retry up to 5 times
3. **Chassis Position Adjustment:**
 - If the target X coordinate exceeds `waste_grasp_distance[0]`, move the chassis forward/backward
 - If the target Y coordinate exceeds `waste_grasp_distance[1]`, move the chassis left/right
 - Wait 4 seconds after each movement to ensure camera stability
4. **Re-acquire Pose:** If the chassis moved, re-call `_get_target_pose()` to get updated 3D coordinates
5. **Calculate Gripper Rotation Angle:**
 - Calculate joint5 (wrist yaw) adjustment angle based on the detection box `angle`
 - If absolute angle > 5°, then `arm_joint5_angle = 90 + angle` (90° is neutral position)
 - Otherwise, maintain neutral position (90°)

6. **Execute Grasp:** Call `_waste_grasp()` to execute the actual grasping action, passing 3D coordinates and joint angles
7. **Auto Backward:** After grasping, move backward by `auto_back_dist` distance

Key Technical Points:

- **Angle Compensation:** Adjust gripper direction based on the OBB detection box rotation angle to ensure optimal grasping of irregular objects
- **Chassis Coordination:** Automatically adjust chassis position when the target is out of working range, ensuring the robotic arm can reach it
- **Multiple Retries:** Retry up to 5 times when acquiring pose to improve success rate

5.3 PlaceWaste — Place Waste

```
def Placewaste(self):  
    '''Place the grasped waste into the bin'''  
    self.SingleJoint_pub.publish(ArmJoint(id=2, joint=135, time=1200))  
    self.SingleJoint_pub.publish(ArmJoint(id=4, joint=50, time=1200))  
    time.sleep(2.0)  
    self._pubSix_Arm(self.waste_place_pose)  
    time.sleep(3.0)  
    self.SingleJoint_pub.publish(ArmJoint(id=6, joint=0, time=1500))  
    time.sleep(2.0)  
    self._pubSix_Arm(self.init_arm_pose)  
  
    self.move_by_odom(direction='backward', distance=self.auto_back_dist)  
    return True
```

Process Description:

1. **Adjust Shoulder Joint:** joint2 moves to 135°, raise the robotic arm, wait 1.2 seconds
2. **Adjust Elbow Joint:** joint4 moves to 50°, prepare placing pose, wait 1.2 seconds
3. **Move to Placing Pose:** Call `_pubSix_Arm(self.waste_place_pose)` to move to the full placing pose, wait 3 seconds
4. **Release Object:** joint6 (gripper) opens to 0°, release the waste, wait 1.5 seconds
5. **Return to Initial Pose:** Call `_pubSix_Arm(self.init_arm_pose)` to return the robotic arm to a safe position
6. **Auto Backward:** After placing, move backward by `auto_back_dist` distance

Action Group Design:

- The placing process uses step-by-step action groups to avoid objects falling or collision due to rapid robotic arm movement
- Waiting time is set after each joint movement to ensure smooth completion

5.6 waste_classify Detection Node — YOLO Inference Thread

Source path:

```
$HOME/M3Pro_ws/src/waste_classify/waste_classify/waste_classify.py
```

- Waste recognition detection thread inference_worker
-

```

def inference_worker(model_config:Model_Configure,
                    frame_shm_name,
                    control_queue,
                    result_queue,
                    frame_shape,
                    ready_event,
                    show_image=False):
    """Inference subprocess"""
    from ultralytics import YOLO
    from ultralytics.engine.results import Results
    try:
        yolo_model = YOLO(model_config.model_path,task="obb")
    except Exception as e:
        print(Fore.RED+f"YOLO Model initialization error :{e} "+Fore.RESET)
        return False

    frame_shm = shared_memory.SharedMemory(name=frame_shm_name)
    frame_array = np.ndarray(frame_shape, dtype=np.uint8, buffer=frame_shm.buf)
    print(Fore.GREEN+'Subprocess: model loaded, waiting for inference
commands...' +Fore.RESET)
    ready_event.set()

    while True:
        try:
            cmd = control_queue.get(timeout=0.1)
            if cmd == 'exit':
                print(Fore.YELLOW+'Subprocess: exit command
received'+Fore.RESET)
                break
            elif cmd == 'infer':
                frame = frame_array.copy()
                results:list[Results] = yolo_model(frame,
                                                  conf=model_config.conf,
                                                  rect=model_config.rect,
                                                  vid_stride=model_config.vid_stride,
                                                  max_det=model_config.max_det,
                                                  half=model_config.half,
                                                  augment=model_config.augment,
                                                  verbose=model_config.verbose,
                                                  classes=model_config.classes,
                                                  device=model_config.device
                                                  )

                waste_items = []
                annotated_frame = frame
                for result in results:
                    if result.obb is None or len(result.obb) == 0:
                        continue
                    annotated_frame = result.plot()

                    xywhr = result.obb.xywhr
                    xyxyxyxy = result.obb.xyxyxyxy
                    names = [result.names[cls.item()] for cls in
result.obb.cls.int()]
                    confs = result.obb.conf

                    for i in range(len(names)):
                        item = {
                            'class_name': names[i],

```

```

        'confidence': float(confs[i]),
        'center_x': float(xywhr[i][0]),
        'center_y': float(xywhr[i][1]),
        'width': float(xywhr[i][2]),
        'height': float(xywhr[i][3]),
        'angle': float(xywhr[i][4]),
        'points': xyxyxyxy[i].tolist()
    }
    waste_items.append(item)

    if show_image:
        cv2.imshow('Inference Result', annotated_frame)
        cv2.waitKey(1)

    # Always put result to queue, even if empty
    result_queue.put((waste_items, annotated_frame))
except:
    pass
cv2.destroyAllWindows()
frame_shm.close()

```

Code Description:

1. **Model Loading:** Load the YOLO OBB model using the ultralytics library (`task="obb"`)
2. **Shared Memory:** Share camera images with the main process via `shared_memory`, avoiding data copying
3. **Inference Loop:**
 - Receive control commands from `control_queue` ('exit' to quit, 'infer' for inference)
 - Upon receiving 'infer' command, perform inference on the current frame
4. **Result Parsing:**
 - Extract OBB detection box `xywhr` (center point, width, height, angle)
 - Extract 4 vertex coordinates `xyxyxyxy`
 - Extract category names and confidence
5. **Result Return:** Put detection results and annotated frame into `result_queue`

YOLO Inference Parameters:

Parameter	Description	Default
<code>conf</code>	Confidence threshold, results below this value are filtered	0.1
<code>rect</code>	Whether to use rectangular input (improves speed)	True
<code>vid_stride</code>	Video frame skip stride (1 means detect every frame)	1
<code>max_det</code>	Maximum number of detection targets	20
<code>half</code>	Whether to use half-precision FP16 inference (improves speed)	True
<code>augment</code>	Whether to use test-time augmentation (improves accuracy)	True
<code>classes</code>	Detection category list (0-15 corresponds to 16 garbage types)	[0,1,2,...,15]
<code>device</code>	Inference device (cuda:0 for GPU)	"cuda:0"

6. Parameter Tuning

- Parameter file path:

```
$HOME/M3Pro_ws/src/m3pro_bringup/config/common.yaml
```

6.1 Garbage Sorting Grasp Parameters

```
mcp_service:
  ros__parameters:
    waste_grasp_offset: [0.0, 0.015, -0.02, 1.57] # Garbage sorting grasp
    offset
    waste_grasp_distance: [0.24, 0.09] # Garbage sorting grasp
    distance threshold
    waste_place_pose: [90, 50, 90, 50, 90, 140] # Garbage sorting placing
    pose
    waste_detect_pose: [90, 90, 50, -30, 90, 0] # Garbage sorting detection
    pose
```

Detailed Parameter Description:

Parameter	Type	Description	Default	Adjustment Suggestion
<code>waste_grasp_offset</code>	[x, y, z, angle]	Grasp position offset, used for fine-tuning grasp coordinates and gripper angle	[0.0, 0.015, -0.02, 1.57]	Adjust based on actual grasp 偏差, $\pm 0.005\text{m}$ each time
<code>waste_grasp_distance[0]</code>	float	X-direction grasp distance threshold (meters), triggers chassis forward/backward adjustment	0.24	Increase to reduce chassis movement frequency
<code>waste_grasp_distance[1]</code>	float	Y-direction grasp distance threshold (meters), triggers chassis left/right adjustment	0.09	Increase to reduce chassis movement frequency
<code>waste_place_pose</code>	[j1, j2, j3, j4, j5, j6]	6-joint target angles for waste placement (degrees)	[90, 50, 90, 50, 90, 140]	Adjust according to trash bin height and position
<code>waste_detect_pose</code>	[j1, j2, j3, j4, j5, j6]	6-joint target angles for waste detection (degrees)	[90, 90, 50, -30, 90, 0]	Ensure the camera can fully capture the desktop

waste_grasp_offset Adjustment Principles:

Unlike normal grasping logic, garbage sorting grasping needs to consider the object's rotation angle:

1. **X-axis Offset (forward/backward):** Usually 0.0, distance is ensured by chassis adjustment
2. **Y-axis Offset (left/right):** Default 0.015m, compensates for lateral offset between camera and gripper
3. **Z-axis Offset (up/down):** Default -0.02m, slightly lowers grasp height to ensure gripping the object
4. **Grasp Angle (4th parameter):** Default 1.57 radians (90°), indicates the gripper is perpendicular to the desktop

6.2 YOLO Model Detection Parameters

```
waste_detect:
  ros__parameters:
    detect_enable: False           # Detection disabled by default at startup
    model_path: ''                # YOLO model path
    conf: 0.1                     # Confidence threshold
    vid_stride: 1                 # Frame skip stride
```

```
max_det: 20                # Maximum detection count
rect: True                 # Rectangular input
half: True                 # Half-precision inference
augment: True              # Test-time augmentation
verbose: False             # Detailed logging
classes: [0,1,2,...,15]    # Detection categories (16 garbage types)
device: "cuda:0"           # Inference device
show_image: True           # Show detection window
image_save_path: ''        # Recognition image save path
auto_shutdown_time: 240.0  # Auto shutdown time (seconds)
always_on: False           # whether to keep always on
```

Detailed Parameter Description:

Parameter	Type	Description	Default	Adjustment Suggestion
<code>detect_enable</code>	bool	Enable detection at startup	False	Suggested False, enable on demand to save resources
<code>conf</code>	float	Confidence threshold (0.0~1.0)	0.1	Increase to reduce false positives, but may miss detections
<code>max_det</code>	int	Maximum targets per detection	20	Adjust based on actual scenario
<code>half</code>	bool	Use FP16 half-precision	True	Recommended for GPU inference to improve speed
<code>augment</code>	bool	Test-time data augmentation	True	Improves accuracy but reduces speed, can disable for faster performance
<code>classes</code>	list	Detection category ID list	[0-15]	Can narrow scope to improve speed
<code>device</code>	string	Inference device	"cuda:0"	Change to "cpu" when no GPU is available
<code>auto_shutdown_time</code>	float	Auto shutdown time after no operation (seconds)	240.0	Adjust based on usage frequency
<code>show_image</code>	bool	Show detection window	True	Can disable for remote debugging

Performance Optimization Suggestions:

- If detection speed is slow, try: disable `augment`, reduce `max_det`, use `half=True`
- If false positives are too many, increase `conf` to 0.2~0.3

- If missed detections are too many, decrease `conf` to 0.05~0.08, or enable `augment`